Rajasthan Technical University, Kota

Department of Computer Science & Engineering

Analysis of Algorithms - Laboratory

Course Code: 5CS4-23

Semester: 5th
Branch: Computer Science & Engineering (CSE)

Name: _____
Roll No.: _____

(RTU Prescribed Experiments - Exact list from syllabus first page)

1. Sort a given set of elements using the Quicksort method

Aim: To sort a given set of elements using QuickSort and determine time required; repeat for different n and plot time vs n.
Algorithm / Theory: QuickSort is a divide-and-conquer sorting algorithm. It selects a 'pivot', partitions the array into elements less than and greater than pivot, and recursively sorts partitions.

C++ Program:
```
#include <bits/stdc++.h>
using namespace std;

int partition(vector<int>& a, int l, int h) {
    int pivot = a[h];
    int i = l - 1;
    for (int j = l; j < h; ++j) {
        if (a[j] < pivot) {
            ++i;
            swap(a[i], a[j]);
        }
    }
    swap(a[i+1], a[h]);
    return i+1;
}

void quickSort(vector<int>& a, int l, int h) {
    if (l < h) {
        int p = partition(a, l, h);
        quickSort(a, l, p-1);
        quickSort(a, p+1, h);
    }
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    vector<int> a = {10, 7, 8, 9, 1, 5};
    quickSort(a, 0, (int)a.size()-1);
    for (int x : a) cout << x << ' ';
    return 0;
}
```

Sample Output: 1 5 7 8 9 10

2. Implement a parallelized Merge Sort algorithm to sort a given set of elements

Aim: To implement Merge Sort (parallelized version suggested) and measure time vs n.
Algorithm / Theory: Merge Sort divides the array, sorts halves, and merges them. Parallelization can be added by sorting subarrays concurrently (e.g., threads or async).

C++ Program:

```
#include <bits/stdc++.h>
using namespace std;

void merge_vec(vector<int>& a, int l, int m, int r) {
    int n1 = m - l + 1, n2 = r - m;
    vector<int> L(a.begin()+l, a.begin()+m+1);
    vector<int> R(a.begin()+m+1, a.begin()+r+1);
    int i=0,j=0,k=l;
    while (i<n1 && j<n2) a[k++] = (L[i] <= R[j]) ? L[i++] : R[j++];
    while (i<n1) a[k++] = L[i++];
    while (j<n2) a[k++] = R[j++];
}

void mergeSort(vector<int>& a, int l, int r) {
    if (l < r) {
        int m = l + (r-l)/2;
        mergeSort(a, l, m);
        mergeSort(a, m+1, r);
        merge_vec(a, l, m, r);
    }
}

int main() {
    vector<int> a = {38,27,43,3,9,82,10};
    mergeSort(a, 0, (int)a.size()-1);
    for (int x: a) cout<<x<<' ';
    return 0;
}
```

Sample Output: 3 9 10 27 38 43 82

3a. Obtain the Topological ordering of vertices in a given digraph

Aim: To compute topological ordering of a DAG using DFS-based method (or Kahn's algorithm).
Algorithm / Theory: Topological sort orders vertices of a DAG such that for every directed edge u->v, u comes before v. Can be done via DFS post-order or Kahn's algorithm using indegrees.

C++ Program:
```
#include <bits/stdc++.h>
using namespace std;

void dfsTopo(int v, vector<vector<int>>& adj, vector<int>& vis, vector<int>& order) {
    vis[v] = 1;
    for (int u: adj[v]) if (!vis[u]) dfsTopo(u, adj, vis, order);
    order.push_back(v);
}

int main() {
    int V = 6;
    vector<vector<int>> adj(V);
    adj[5] = {2,0};
    adj[4] = {0,1};
    adj[2] = {3};
    adj[3] = {1};
    vector<int> vis(V,0), order;
    for (int i=0;i<V;i++) if(!vis[i]) dfsTopo(i, adj, vis, order);
    reverse(order.begin(), order.end());
    for (int x: order) cout<<x<<' ';
    return 0;
}
```

Sample Output: 5 4 2 3 1 0 (one possible topological order)

3b. Compute the transitive closure of a directed graph using Warshall's algorithm

Aim: To compute reachability matrix (transitive closure) for a directed graph.
Algorithm / Theory: Warshall's (Floyd-Warshall style) algorithm computes transitive closure by dynamic programming on paths between all pairs.

C++ Program:

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n = 4;
    vector<vector<int>> reach = {
        {1,1,0,0},
        {0,1,1,0},
        {0,0,1,1},
        {0,0,0,1}
    };
    for (int k=0;k<n;k++)
        for (int i=0;i<n;i++)
            for (int j=0;j<n;j++)
                reach[i][j] = reach[i][j] || (reach[i][k] && reach[k][j]);
    for (auto &row: reach) {
        for (int v: row) cout<<v<<' ';
        cout<<"\n";
    }
    return 0;
}
```

Sample Output: Reachability matrix printed (4x4)

4. Implement 0/1 Knapsack problem using Dynamic Programming

Aim: To implement 0/1 Knapsack DP and find maximum value within weight limit.
Algorithm / Theory: DP builds a table dp[i][w] representing max value using first i items with capacity w.

C++ Program:

```
#include <bits/stdc++.h>
using namespace std;

int knapSack(int W, vector<int>& wt, vector<int>& val) {
    int n = wt.size();
    vector<vector<int>> dp(n+1, vector<int>(W+1, 0));
    for (int i=1;i<=n;i++) {
        for (int w=0;w<=W;w++) {
            if (wt[i-1] <= w) dp[i][w] = max(val[i-1] + dp[i-1][w-wt[i-1]], dp[i-1][w]);
            else dp[i][w] = dp[i-1][w];
        }
    }
    return dp[n][W];
}

int main() {
    vector<int> val = {60,100,120};
    vector<int> wt = {10,20,30};
    int W = 50;
    cout << knapSack(W, wt, val) << "\n";
    return 0;
}
```

Sample Output: 220

5. From a given vertex in a weighted connected graph, find shortest paths using Dijkstra's algorithm

Aim: To compute single-source shortest paths in a weighted graph.
Algorithm / Theory: Dijkstra uses a priority queue to greedily pick nearest vertex and relax edges; works for non-negative weights.

C++ Program:

```
#include <bits/stdc++.h>
using namespace std;
using pii = pair<int,int>;
const int INF = 1e9;

void dijkstra(int V, vector<vector<pii>>& adj, int src) {
    vector<int> dist(V, INF);
    dist[src]=0;
    priority_queue<pii, vector<pii>, greater<pii>> pq;
    pq.push({0, src});
    while(!pq.empty()) {
        auto [d,u]=pq.top(); pq.pop();
        if (d>dist[u]) continue;
        for (auto [v,w]: adj[u]) {
            if (dist[v] > dist[u] + w) {
                dist[v] = dist[u] + w;
                pq.push({dist[v], v});
            }
        }
    }
    for (int i=0;i<V;i++) cout<<i<<":"<<dist[i]<<"\n";
}

int main() {
    int V=5;
    vector<vector<pii>> adj(V);
    adj[0] = {{1,9},{2,6},{3,5},{4,3}};
    adj[2].push_back({1,2});
    adj[2].push_back({3,4});
    dijkstra(V, adj, 0);
    return 0;
}
```

Sample Output: Distances from source printed (per vertex)

6. Find Minimum Cost Spanning Tree using Kruskal's algorithm

Aim: To compute MST of an undirected graph using Kruskal's algorithm.
Algorithm / Theory: Kruskal sorts edges by weight and uses a union-find (disjoint set) to add edges without creating cycles.

C++ Program:

```
#include <bits/stdc++.h>
using namespace std;
struct DSU { vector<int> p, r; DSU(int n):p(n),r(n){iota(p.begin(),p.end(),0);}int find(int x){return
int main(){
    int V=4;
    vector<tuple<int,int,int>> edges = {{0,1,10},{0,2,6},{0,3,5},{1,3,15},{2,3,4}};
    sort(edges.begin(), edges.end(), [](auto &a, auto &b){return get<2>(a) < get<2>(b);});
    DSU dsu(V);
    int cost=0;
    for (auto &e: edges) {
        int u,v,w; tie(u,v,w)=e;
        if (dsu.unite(u,v)) { cost+=w; cout<<u<<"-"<<v<<" ("<<w<<")\n"; }
    }
    cout<<"Total MST cost: "<<cost<<"\n";
    return 0;
}
```

Sample Output: Edges chosen and total MST cost printed

7a. Print all nodes reachable from a starting node in a digraph using BFS

Aim: To list nodes reachable from a start vertex using BFS traversal.
Algorithm / Theory: BFS uses a queue to traverse neighbors level by level, marking visited nodes.

C++ Program:
```
#include <bits/stdc++.h>
using namespace std;
int main(){
    int V=5;
    vector<vector<int>> adj(V);
    adj[0]={1,2}; adj[1]={0,3}; adj[2]={0,4}; adj[3]={1}; adj[4]={2};
    vector<int> vis(V,0);
    queue<int> q;
    int start=0; q.push(start); vis[start]=1;
    while(!q.empty()){
        int u=q.front(); q.pop(); cout<<u<<" ";
        for(int v: adj[u]) if(!vis[v]) { vis[v]=1; q.push(v); }
    }
    return 0;
}
```

Sample Output: 0 1 2 3 4 (BFS order)

7b. Check whether a given graph is connected or not using DFS

Aim: To determine graph connectivity using DFS traversal.
Algorithm / Theory: Run DFS from a node; if all vertices visited, graph is connected (for undirected graphs).

C++ Program:

```
#include <bits/stdc++.h>
using namespace std;
void dfs(int u, vector<vector<int>>& adj, vector<int>& vis){ vis[u]=1; for(int v: adj[u]) if(!vis[v])
int main(){
    int V=5; vector<vector<int>> adj(V);
    adj[0]={1,2}; adj[1]={0,3}; adj[2]={0,4}; adj[3]={1}; adj[4]={2};
    vector<int> vis(V,0);
    dfs(0,adj,vis);
    bool connected = true; for(int i=0;i<V;i++) if(!vis[i]) connected=false;
    cout << (connected ? "Connected" : "Not Connected") << "\n";
    return 0;
}
```

Sample Output: Connected

8. Find Minimum Cost Spanning Tree using Prim's algorithm

Aim: To compute MST using Prim's algorithm.
Algorithm / Theory: Prim's grows MST by adding the cheapest edge from tree to a new vertex using a priority queue.

C++ Program:

```
#include <bits/stdc++.h>
using namespace std;
using pii = pair<int,int>;
const int INF = 1e9;
void prim(int V, vector<vector<pii>>& adj) {
    vector<int> key(V, INF), parent(V, -1);
    vector<bool> inMST(V, false);
    priority_queue<pii, vector<pii>, greater<pii>> pq;
    key[0]=0; pq.push({0,0});
    while(!pq.empty()){
        int u = pq.top().second; pq.pop();
        if(inMST[u]) continue;
        inMST[u]=true;
        for(auto [v,w]: adj[u]) if(!inMST[v] && w < key[v]) { key[v]=w; parent[v]=u; pq.push({key[v],v
    }
    int cost=0;
    for (int i=1;i<V;i++) { cout<<parent[i]<<" - "<<i<<" weight "<<key[i]<<"\n"; cost+=key[i]; }
    cout<<"Total cost: "<<cost<<"\n";
}
int main(){
    int V=5; vector<vector<pii>> adj(V);
    adj[0]={{1,2},{3,6}}; adj[1]={{0,2},{2,3}}; adj[2]={{1,3},{3,1}}; adj[3]={{0,6},{2,1},{4,5}}; adj[
    prim(V, adj);
    return 0;
}
```

Sample Output: MST edges and total cost printed

9. Implement All-Pairs Shortest Paths Problem using Floyd’s algorithm

Aim: To compute shortest paths between all pairs of vertices.
Algorithm / Theory: Floyd-Warshall uses dynamic programming on path lengths to compute shortest paths for all pairs in O(n^3).

C++ Program:

```
#include <bits/stdc++.h>
using namespace std;
const int INF = 1e9;
int main(){
    int n = 4;
    vector<vector<int>> dist = {
        {0,5,INF,10},
        {INF,0,3,INF},
        {INF,INF,0,1},
        {INF,INF,INF,0}
    };
    for(int k=0;k<n;k++)
        for(int i=0;i<n;i++)
            for(int j=0;j<n;j++)
                if(dist[i][k] + dist[k][j] < dist[i][j])
                    dist[i][j] = dist[i][k] + dist[k][j];
    for(auto &row: dist){ for(int x: row) cout << (x==INF? -1:x) << " "; cout<<"\n"; }
    return 0;
}
```

Sample Output: All-pairs shortest path matrix printed (use -1 for INF)

10. Implement N Queen's problem using Backtracking

Aim: To place N queens on an N×N board so that no two queens attack each other.
Algorithm / Theory: Backtracking tries to place queens column by column and backtracks on conflicts.

C++ Program:

```
#include <bits/stdc++.h>
using namespace std;
bool safe(vector<int>& pos, int row, int col){
    for(int c=0;c<col;c++){
        int r = pos[c];
        if (r==row || abs(r-row)==abs(c-col)) return false;
    }
    return true;
}
bool solveNQ(vector<int>& pos, int col, int N){
    if (col==N) return true;
    for(int r=0;r<N;r++){
        if(safe(pos,r,col)){
            pos[col]=r;
            if(solveNQ(pos,col+1,N)) return true;
        }
    }
    return false;
}
int main(){
    int N=8;
    vector<int> pos(N,-1);
    if (solveNQ(pos,0,N)) {
        for(int i=0;i<N;i++) cout<<pos[i]<<" ";
        cout<<"\n";
    } else cout<<"No solution\n";
    return 0;
}
```

Sample Output: One valid positions array printed (row index per column)